

#Admin

Accept/Reject Applicant Workflow

Summary

Add protected POST actions for pending applications. Accepting an application will mark it `ACCEPTED`, create exactly one `student\_detail` record, and refresh the dashboard's existing Accepted count. Rejecting will mark it `REJECTED`.

Key Changes

- Change `student\_detail.student\_id` and linked `student\_login.student\_id` from `INT` to `VARCHAR` so IDs are stored exactly as `scc0001`, `scc0002`, and so on. Update the foreign-key migration safely by recreating the affected constraint.
- Add `POST` endpoints and register them in `main.py` for accepting and rejecting a specific `application\_id`.
- On accept, use one database transaction to:
 - Lock and fetch the pending application plus its department name.
 - Generate the next unused `scc` ID, padded to at least four digits.
 - Update `student\_admission.status` to `ACCEPTED`.
 - Insert all requested values into `student\_detail`, with `status = ACCEPTED`, `joined\_at = CURRENT\_TIMESTAMP`, `department\_name` from `department`, and `class\_no = 1`.
- Prevent duplicate student records by checking the existing unique `application\_id` relationship.
- On reject, update only pending applications to `REJECTED`; no `student\_detail` record is created.
- Replace the placeholder button forms in `admin.html` with application-specific POST forms. Show Accept and Reject only while status is `PENDING`; final decisions remain visible but cannot be changed.

- Redirect back to the applicant section after each action and show success/error messages.

Test Plan

- Accept the first pending application and verify `student\_detail.student\_id = scc0001`.
- Accept another application and verify the ID increments to `scc0002`.
- Verify all requested `student\_detail` fields are populated, including department name, class number `1`, and joined time.
- Verify Accepted and Rejected dashboard counts update after refreshing.
- Verify accepting the same application twice does not create a duplicate student.
- Verify rejected applications do not create student records and finalized rows no longer show action buttons.

Assumptions

- Student IDs must be stored as text exactly in the `scc0001` format.
- Every accepted student initially receives `class\_no = 1`.
- Only pending applications can be accepted or rejected.

Student detail Workflow

Summary

The **Student Detail** module is used by the administrator to display and search registered student records in the Student Enrollment System.

When an admission application is accepted, the system creates a corresponding record in the student_detail table. A unique Student ID is generated, the applicant information is transferred, and the student's initial status is set to **CONTINUE**.

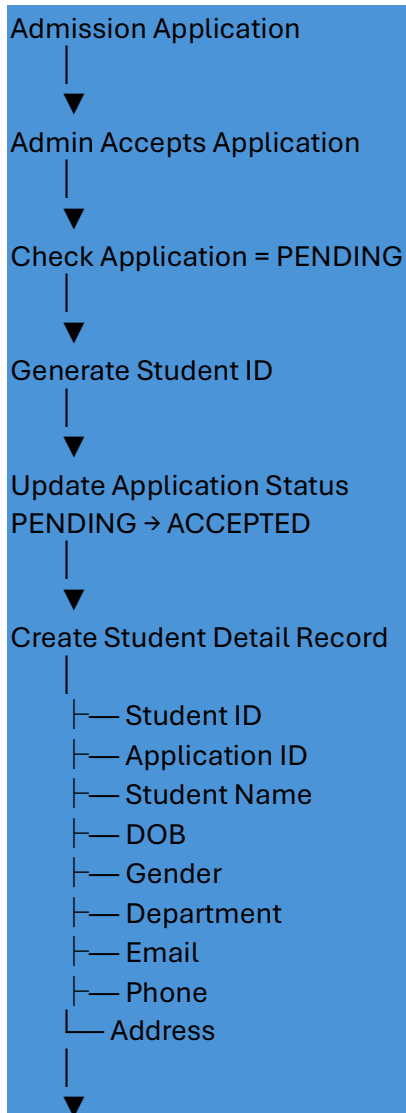
The Admin Students page then retrieves these records and allows the administrator to:

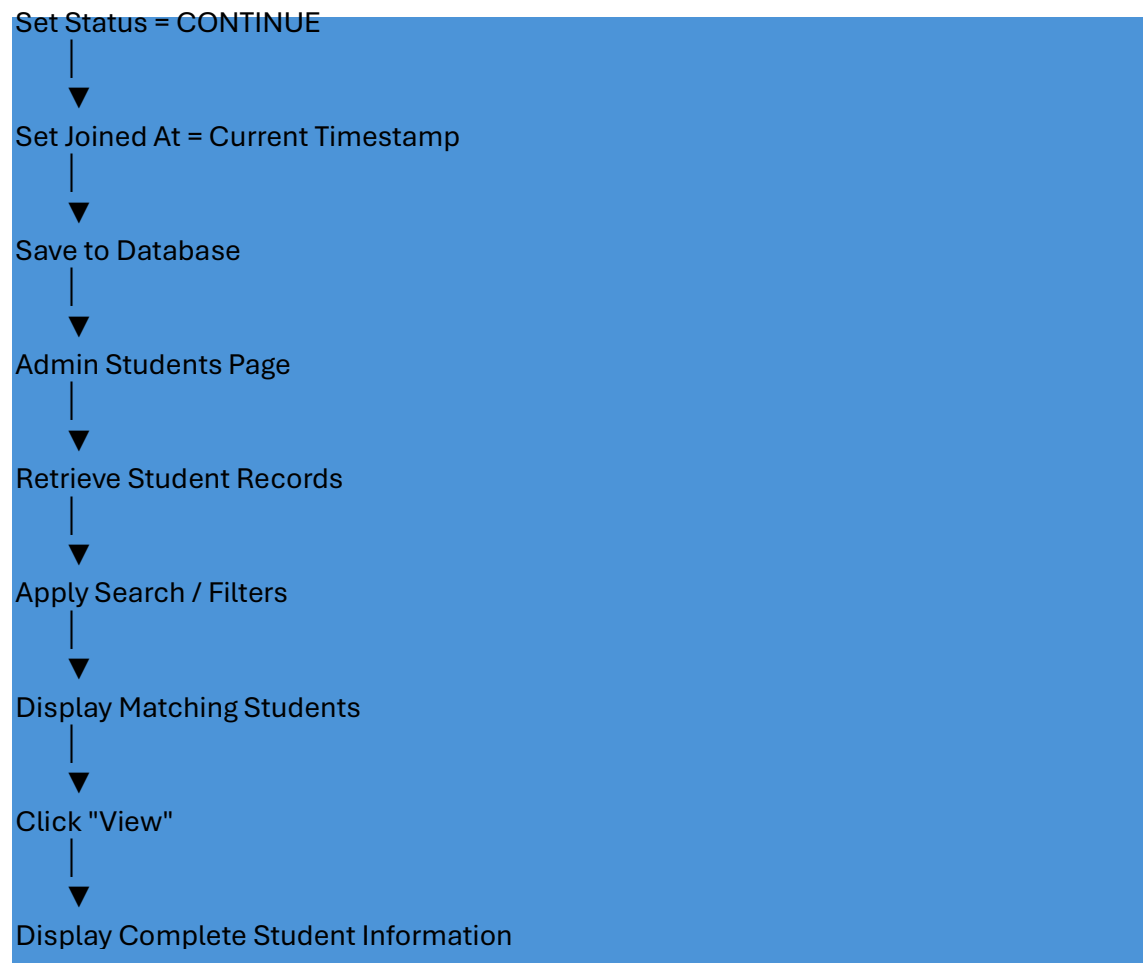
- Search by **Student ID**

- Search by **Application ID**
- Search by **Student Name**
- Filter by **Department**
- Filter by **Student Status**
- Filter by **Joined Date**
- View complete student information

The student records are retrieved by the `admin_student_detail()` Python function and displayed through the `student_records` variable in the HTML template.

Student Detail Workflow





Key Changes / Functionality

Student Record Creation

When an application is accepted, the system creates a student record rather than requiring the administrator to manually enter the student again.

The application information is copied from student_admission into student_detail.

Automatic Student ID

The system automatically generates a Student ID using the format:

scc0001

scc0002

scc0003

...

The latest numeric Student ID is identified and incremented to generate the next ID. A database lock is also used during this process.

Student Status

A newly created student record is assigned:

CONTINUE

The display/filter logic also maps existing NEW and ACCEPTED student-detail statuses to CONTINUE.

Student Information Display

The Student Details table displays:

Field	Purpose
Student ID	Unique student identifier
Application ID	Links the student to the application
Student Name	Student's name
Email	Student email
Phone	Contact number
Status	Current student status
Joined At	Student joining date
Actions	View complete details

Filter Workflow

The Student Detail filter is submitted using a **GET request** to the Admin Dashboard.





Filter Details

Student ID / Application ID / Name Search

The **Search Student** field accepts one search value.

Example:

scc0001

or

1001

or

Arun

The backend searches three fields:

student_id

OR

application_id

OR

student_name

The search uses LIKE, so partial values can be matched.

Example

Searching:

scc00

can return matching Student IDs such as:

scc0001

scc0002

scc0003

Department Filter

The administrator can select a department from the dropdown.

All Departments

BCA

BSc Computer Science

B.Com

BSc Physics

...

The selected department ID is passed to the backend and used in:

AND sd.department_id = %s

Status Filter

The available student statuses are:

All

Continue

Discontinue

The backend converts the selected status to uppercase before processing.

The query then filters the student record based on the normalized status.

Joined Date Filter

The administrator can select a specific joining date.

Example:

2026-08-25

The backend compares this value with the date portion of joined_at:

DATE(sd.joined_at) = %s

Therefore, records joined on that particular date are displayed.

#. Combined Filtering

Multiple filters can be used together.

For example:

Search Student: Arun

Department: BCA

Status: Continue

Joined Date: 2026-08-25

The system applies all selected conditions:

Student Name / ID / Application ID

+

Department

+

Status

+

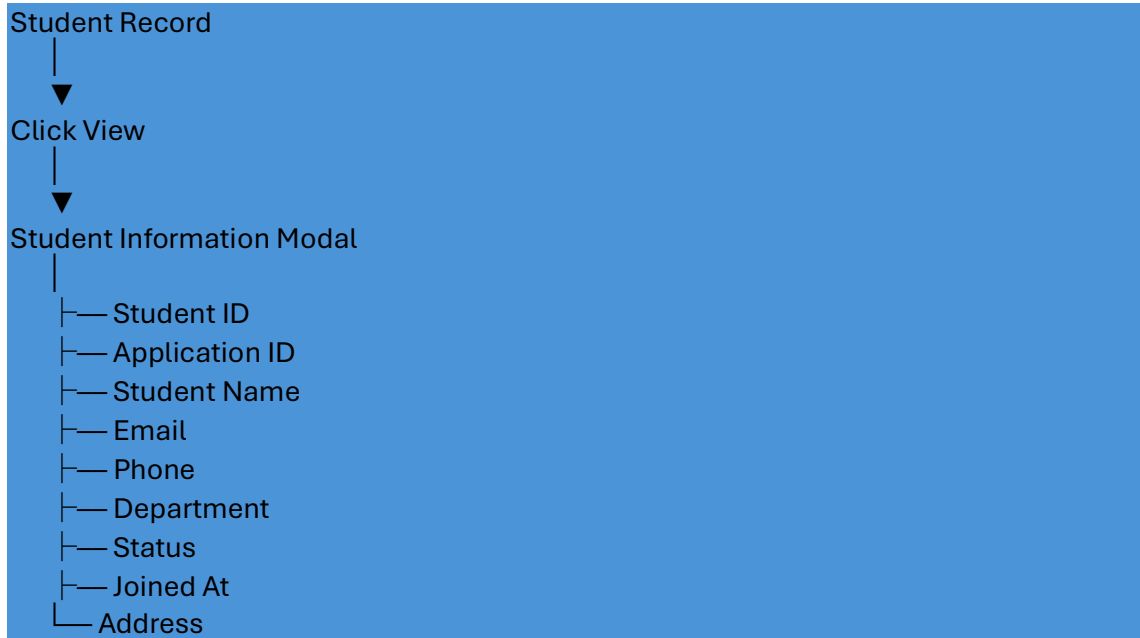
Joined Date

Only matching records are returned.

If a filter is empty, that condition is not added to the SQL query.

View Student Details

After the filtered records are displayed, each record contains a **View** button.



Clear Filter Workflow

The Student Details section also contains a **Clear** button.



Database Workflow

The main tables involved are:



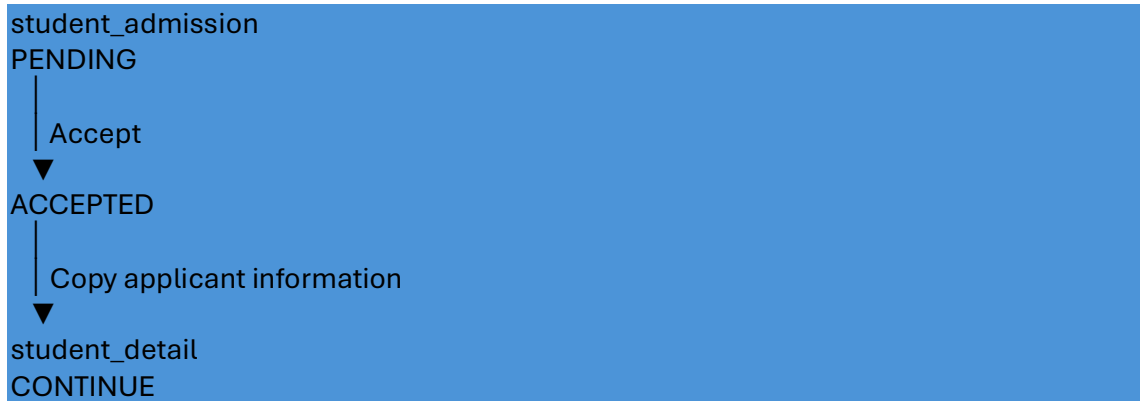
student_admission

Contains the application submitted by the applicant.

student_detail

Contains the student record after admission acceptance.

The acceptance operation:



#Test Plan

Student Record Creation Tests

Test ID	Test Case	Expected Result
SD-01	Accept a pending application	Application becomes ACCEPTED
SD-02	Accept application	Student record is created
SD-03	Accept application	Unique Student ID is generated
SD-04	Check new student status	Status is CONTINUE
SD-05	Check student information	Applicant information is copied correctly
SD-06	Try accepting already finalized application	System prevents another decision
SD-07	Verify joined date	Joined timestamp is created

10.2 Search Tests

Test ID	Test Case	Expected Result
SD-08	Search by Student ID	Matching student displayed
SD-09	Search by Application ID	Matching student displayed
SD-10	Search by Student Name	Matching students displayed
SD-11	Search using partial value	Matching records displayed
SD-12	Search invalid value	"No student details found" displayed

10.3 Department Filter Tests

Test ID	Test Case	Expected Result
SD-13	Select BCA	Only BCA students displayed
SD-14	Select another department	Only selected department displayed
SD-15	Select All Departments	Students from all departments displayed

10.4 Status Filter Tests

Test ID	Test Case	Expected Result
SD-16	Select Continue	Continue students displayed
SD-17	Select Discontinue	Discontinued students displayed
SD-18	Select All	All student statuses displayed

10.5 Joining Date Tests

Test ID	Test Case	Expected Result
SD-19	Select valid joining date	Students joined on that date displayed
SD-20	Select date with no students	No student details found
SD-21	Leave date empty	Date filtering not applied

10.6 Combined Filter Tests

Test ID	Test Case	Expected Result
SD-22	Name + Department	Matching students displayed
SD-23	Department + Status	Matching students displayed
SD-24	Department + Date	Matching students displayed
SD-25	Search + Department + Status + Date	Only records matching all conditions displayed
SD-26	Clear all filters	Filter values are cleared and normal list is restored

10.7 View Details Tests

Test ID	Test Case	Expected Result
SD-27	Click View	Student Information modal opens
SD-28	Check modal information	Correct student information displayed
SD-29	Close modal	Modal closes successfully

11. Assumptions

The following assumptions are based on the uploaded HTML and Python implementation:

1. The administrator must be logged in before accessing the Student Detail section.
2. A student record is created when a pending application is accepted.
3. The application must have PENDING status before the administrator can accept it.
4. Each student has a unique Student ID.
5. Student IDs use the scc prefix followed by a numeric sequence.
6. A newly created student is assigned CONTINUE status.
7. joined_at stores the student's joining timestamp.

8. The department information is available through the department data used by the application.
9. Student search supports Student ID, Application ID, and Student Name through one search box.
10. Department, status, and joined date are separate filters.
11. Multiple filters can be applied at the same time.
12. Student details are retrieved from the student_detail table.
13. The **Clear** button's exact behavior depends on the JavaScript in script.js, which was not included in the uploaded files.
14. The database connection is configured through the values imported from config.py.

#Final Workflow

Stage	Process	Result
1	Admin accepts application	Application accepted
2	Generate Student ID	Unique ID created
3	Copy applicant information	Student data prepared
4	Insert into student_detail	Student record created
5	Set status	CONTINUE
6	Set joined timestamp	Joining date stored
7	Open Admin Students	Student records displayed
8	Search	Find by ID, Application ID or name
9	Filter	Department, status and joined date
10	Combine filters	Narrow results
11	View	Complete student information displayed